uni-mem：面向异构编码智能体的网关式统一记忆层

技术报告（v0.3，2026 年 10 月）

摘要

主流编码智能体（coding harness），如 Claude Code、Codex CLI，各自带有长期记忆机制，但这些记忆格式不同、存储分离、彼此不可见，形成一个个“记忆孤岛”。基于 MCP 或 hook 的记忆方案虽然可以共享存储，却要为每个 harness 单独集成，而且依赖模型主动调用。本文提出 **uni-mem**：一个运行在本机的单二进制 LLM 网关。所有 harness 只需把模型 API 的 base URL 指向它，就共享同一份按项目自动归档的长期记忆。uni-mem 有三个核心机制：(1) **overlay 重放**，网关对请求的每一处改动都以内容锚点记录，并在同一会话后续请求中逐字节重放，保证 prompt cache 前缀稳定；(2) **流式工具拦截与续写**，对 Anthropic Messages、OpenAI Responses、Chat Completions 三种方言，在网关内执行模型发出的 `unimem_*` 工具调用并续写上游生成，harness 只看到一条连续的回复；(3) 一个借鉴 Jev-Mem System-One/System-Two 划分的**记忆引擎**，用可插拔的类型化决策器处理“是否记、记到哪个作用域、是否覆盖旧记忆、是否相关、证据是否充分”等高频小判断。

所有评测都只用离线启发式决策器和哈希嵌入器，没有真实 LLM 参与，也没有真实 harness 流量。在样本外数据集 heldout2（242 条查询，233 条笔记，3 个 harness）上，v0.3 的 R@5 为 0.688，R@1 为 0.555，MRR 为 0.616，precision 为 0.423，过期笔记返回率为 16.3%。它在 R@5 上显著优于 v0.1（+0.119）、v0.2（+0.138）和 bm25-top5-cwd（+0.073），但**并未显著优于**使用同一嵌入器的 top-5 余弦检索（+0.018，95% CI [−0.037, +0.073]）。跨 harness 查询上，统一存储的 R@5 为 0.664，按 harness 隔离的 silo 为 0.000。网关流式路径的附加延迟为 p50 0.898 ms / p95 2.361 ms。30 轮对话中 42/42 次上游请求转换都是逐字节前缀扩展；1060/1060 条拦截正确性断言通过。主要负面结果是：写入端的安全筛查没有泛化（样本外提示注入仅拦截 1/7，状态笔记 1/6，而样本内均为 100%）；跨语言检索 R@5 只有 0.333；图扩展和被动门控没有可测量的贡献。

关键词：编码智能体；长期记忆；LLM 网关；prompt cache；工具拦截；System One

---

1 引言

1.1 问题

编码智能体正在成为开发者的日常工具，而且同一个人往往同时使用多个 harness。各 harness 都已有自己的记忆机制：

- **Claude Code** 提供 auto memory。记忆存放在 `~/.claude/projects/<project>/memory` 目录下，以 `MEMORY.md` 为索引；会话开始时加载索引的前 200 行或 25KB。此外还有 `CLAUDE.md` 形式的静态项目指令。
- **Codex CLI** 提供本地 memories（`~/.codex/memories`），由后台抽取与整合流程生成，默认关闭；静态指令则写在 `AGENTS.md`。
- 其它工具大多有各自的规则文件。

这些机制有三个共同问题。第一，它们是**孤岛**：用户在 Claude Code 里纠正过的一件事，Codex 不知道。第二，它们大多不区分“关于我”和“关于这个仓库”的记忆。第三，`CLAUDE.md` / `AGENTS.md` 是静态指令文件，需要人工维护，不是随交互积累的记忆。

一种自然的解法是把记忆做成 MCP server 或 hook，让多个 harness 共享。但 MCP/hook 体系在各 harness 之间并不兼容，每接入一个 harness 都要单独集成；而且 MCP 模式下记忆是否被使用，完全取决于模型是否主动调用工具，网关无法做被动召回。

1.2 思路

uni-mem 从一个观察出发：**每个 harness 都一定会调用模型 API，而且都允许修改 base URL**。模型 API 是唯一“零适配”的公共接入点。把记忆层放在这里，就能在不改动任何 harness 的前提下为它们提供共享记忆。这个位置也带来

了新的工程约束：网关挂在所有流量前面，不能破坏 prompt cache，不能让 harness 看到它不认识的工具调用，任何异常都必须退化为透传。

1.3 贡献

1. 提出网关式统一记忆的架构，并给出一组从第一性原理推出的设计约束（§3.1）。
2. 设计了**缓存稳定的注入与 overlay 重放**机制：以 harness 原样回传的内容为锚点，并按会话与前一条消息隔离（§3.3）。
3. 设计了覆盖三种 API 方言的**流式工具拦截与续写**状态机，包括混合轮次（网关工具与 harness 工具同时出现）的处理（§3.4）。
4. 实现了一个 Rust 记忆引擎：四视图关系图、可插拔的 System-One 决策器（每个问题都带代码计算的先验）、自适应截断与充分性信号（§3.6），以及确定性硬信号加模型软信号的项目识别（§3.7）。
5. 给出一套刻意区分样本内/样本外的评测，并如实报告负面结果，包括 v0.2 “修复即过拟合”的教训（§5）。

---

2 相关工作

Harness 内置记忆。 Claude Code 的 auto memory 与 `CLAUDE.md`、Codex 的 memories 与 `AGENTS.md`（见 §1.1）都绑定在单个 harness 的格式与目录中。uni-mem 不替代这些机制，而是在它们之外提供一个跨 harness 的共享层。

MCP 记忆服务器。 jevmem、memsearch、mimir 等项目以 MCP server 形式提供记忆的存取工具。它们可以跨 harness 共享存储，但需要每个 harness 配置 MCP，并依赖模型主动调用。uni-mem 也提供 MCP 端点作为退路（§4），但主路径是网关：网关能看到全部对话，因此可以做被动召回、会话序言和对话流水捕获。

Memory-first 代理。 BladeX 同样采用“在模型 API 前放一个记忆代理”的思路，是同期工作。uni-mem 是独立地从第一性原理设计的，二者在 overlay 重放、工具拦截续写、作用域判定等具体机制上可能存在差异；本文没有与 BladeX 做实验对比。

Jev-Mem 与智能体记忆系统。 Jev-Mem（arXiv 2609.23986）提出把记忆系统中大量的类型化小决策交给快速、便宜、输出概率的 “System One” 模型，阈值写在代码里；环路中的大模型（“System Two”）负责撰写与推理。uni-mem 的记忆引擎直接沿用了这一划分。Jev-Mem 文中讨论的 MAGMA、A-MEM、MemoryOS、Nemori 等系统，在记忆组织（图结构、多视图、分层与整合）方面也影响了本文的数据模型设计；本文没有复现这些系统，也没有与之进行数值比较。

---

3 设计

3.1 第一性原理

设计文档（`docs/DESIGN.md` §0）列出了七条约束：

1. **记忆属于人，不属于工具**。 记忆层必须位于所有 harness 之外，由它们共用。
2. **唯一普适的接入点是模型 API**。 网关是唯一零适配的位置。
3. **网关在本机，因此能看到文件系统**。 项目识别的硬信号（git root、remote、manifest）可以确定性地获得，只有语义部分需要模型判断。
4. **环路里已经有最聪明的模型（System Two）**。 它是最好的记忆撰写者，不必再额外请一个大模型做总结，只需给它工具。
5. **高频小决策交给 System One**。 阈值写在代码和配置里，决策可追踪。
6. **不能让用户为记忆付出缓存代价**。 任何注入要么在整个会话内逐字节稳定，要么位于尾部并在之后原样重放。
7. **失败必须透明**。 记忆层任何异常都退化为纯透传。

3.2 总体架构与 Turn 分类

uni-mem 监听 `127.0.0.1:4777`，暴露 `/v1/messages`（Anthropic）、`/v1/responses`（OpenAI Responses，Codex 使用）、`/v1/chat/completions` 与 `/mcp`。请求依次经过方言适配、Turn 分类、上下文提取（cwd、会话）、overlay 重放、工具注入与项目解析，然后转发上游；响应经流式改写器返回。上游鉴权头（API key 与订阅 OAuth）原样透传。

harness 会发出大量辅助请求，例如标题生成、bash 前缀判定、压缩摘要、quota 探测，这些请求不应注入记忆。分类规则是：

- 请求带 `tools` → **agent turn**：注入工具，执行 overlay 重放；
- 最后一条消息是用户新输入（不是纯 tool_result）→ **prompt turn**：额外执行被动召回；
- `count_tokens`、无工具请求、模型列表等 → 纯透传（`count_tokens` 会附加工具定义，以便 harness 的上下文估算包含它们）。

3.3 缓存稳定的注入与 overlay 重放

Anthropic 与 OpenAI 的 prompt cache 都按前缀匹配。网关对请求的任何非稳定修改都会使后续缓存失效。unimem 的注入位置因此严格受限：

注入内容	位置	稳定性
工具定义	追加到 `tools` 数组末尾	内容固定，逐字节稳定
system prompt	**不修改**	部分订阅鉴权会校验 system 前缀
会话序言（项目卡片 + 置顶记忆）	第一条 user 消息末尾	会话内冻结
被动召回	当前 user 消息末尾（请求尾部）	之后由 overlay 重放

每一处修改都记录为 overlay。overlay 有两类：

- **`augment_user`**：锚点为 `aug_anchor(session, key(前一条消息), key(本消息))`，载荷是追加到该 user 消息后的 `<unimem...>` 文本。空文本表示“已处理、无需追加”，这样同一个 prompt 的重试是幂等的。
- **`expand_assistant`**：锚点为 `exp_anchor(session, key(前一条消息), hash("vis", visible_key(msg)))`。harness 看到的是合并后的一条 assistant 消息，重放时把它整体替换为上游真实产生的序列：`assistant(含 unimem tool_use)` → `user(tool_result)` → `assistant(续写)`。混合轮次中，网关的 tool_result 被前插到下一条 user 消息。

`visible_key` 只取 `text` 块的去空白文本与 `tool_use` 的 id，忽略 thinking、签名、citation 和 cache_control，所以从网关流出的内容与 harness 回传的内容能算出相同的 key。找不到锚点时（历史被压缩或编辑）直接跳过，请求仍然合法，只损失这部分上下文。

会话隔离。 v0.1 的展开锚点只用 `hash(前一条消息 key, 可见 key)`，并以 `ON CONFLICT DO NOTHING` 写入。评测中的锚点碰撞探针发现：两个会话发送相同 prompt 并得到相同可见回复时，第二个会话的下一次请求会被重放第一个会话的隐藏工具调用与结果（10/10）。v0.2 把会话 key 纳入锚点，问题消失（0/10，§5.5）。

3.4 流式工具拦截与续写

网关向模型暴露 `unimem_recall`、`unimem_remember`、`unimem_forget`、`unimem_project` 四个工具。harness 不认识它们，所以这些调用绝不能流到 harness。流式改写器是一个状态机：

1. 边读上游 SSE 边转发，维护 block/item 索引映射；
2. 遇到 `unimem_*` 的 tool_use 块时缓冲而不转发，并重新编号后续块的索引；
3. 上游以 `stop_reason=tool_use` 结束时：
   - **只有网关工具**：吞掉 `message_delta/message_stop`，在网关内执行工具，把 `assistant + tool_result` 拼进请求后再请求上游；跳过新流的 `message_start`，按索引偏移接到原流后面。最多续写 `max_continuations` 轮（默认 6）。
   - **混合轮次**：harness 自己的工具调用照常交给 harness；网关工具在网关内执行，结果存入 overlay，下一次请求时补回。
4. usage 在多轮之间累加，最终的 `message_delta` 给出合并值。

三种方言共享同一套架构，只是粒度不同：

方言	单元	拦截与重放要点
Anthropic Messages	content block	块索引连续重编号；`expand_assistant` 整体替换可见 assistant 消息，保留 thinking 块与其后的工具调用的对应关系
OpenAI Responses	output item	`output_index` 与 `sequence_number` 重编号；最终 `response.completed` 的 `output` 只含可见 item；隐藏 item 以相邻可见 item（call_id、reasoning 的 `encrypted_content` 哈希、消息文本哈希）为锚点插回；`instructions` 不修改
Chat Completions	message / `tool_calls[].index`	网关的 `tool_calls` 被隐藏，其余调用连续重编号；后续轮次文本以空行接续；`n > 1` 的请求直接透传

3.5 失败策略

- 召回超时（recall_budget_ms，默认配置 1500 ms）→ 不注入，直接透传；
- 改写阶段出错 → 退回原始请求；
- 工具执行出错 → 错误作为 tool_result 交给模型自行处理；
- 上游错误 → 状态码与 body 原样返回。

3.6 记忆引擎

数据模型。 Memory 包含作用域（global 或 project:<pid>）、类型（fact / decision / preference / procedure / gotcha / episode）、内容、可选的 why、实体、来源（harness、会话、cwd、git 分支与提交）、superseded_by、显著度、命中次数、置顶标记与嵌入向量。Edge 的关系类型有 semantic、temporal、causal、entity、supersedes、duplicate。此外还有 Project、Episode（无损对话流水，是被动蒸馏的原料）与 Overlay。全部存放在单个 SQLite 文件中。

四视图关系图。 语义、时间、因果、实体四类边构成同一张记忆图的四个视图，另有 supersedes 与 duplicate 两类维护边。代码负责确定性边（如同会话的时间序），其余边由 System One 的概率判断超过阈值后建立。

可插拔 System One。 决策器接口接收一个状态和一批类型化问题（是非题 Noul、选择题 Choice），返回带概率的答案。**每个问题都携带由确定性代码计算的先验**（相似度、正则、关键词、校准过的逻辑回归）。后端有四种：

- JevDecider：TypeSafe /v1/systemone 或兼容服务；
- LlmDecider：任意 OpenAI 兼容小模型，以 JSON 输出模拟类型化问题；
- HeuristicDecider：离线、确定性，直接返回先验；
- ChainDecider：按顺序尝试，前一个不可用就降级。

因此系统在完全没有模型时也能工作，阈值（如 T_INJECTION=0.6、T_DUP=0.8、T_SUPERSEDE=0.7、T_GLOBAL_PREF=0.6）写在代码中，并注明“为启发式先验标定，换决策模型需重新标定”。

写路径。

1. 代码筛查：过短（<8 字符）、过长（>2400 字符）、密钥（厂商格式与“关键词 + 值”两类正则）直接拒绝；
2. 一次批量 System-One 调用，问题包括 durable（是否持久事实而非“下一步做 X”）、injection（是否针对 agent 的提示注入）、kind、scope（在 global / 当前项目 / 文中提到的其它已知项目之间选择，**由模型决定写全局还是写项目**）；
3. 在相关作用域内取候选邻居，嵌入余弦 ⩾ 0.97 且主题一致时直接判为重复；
4. 对候选做批量关系判断：duplicate、supersedes、explains_why（因果）、same_entity。v0.3 的 supersede 与 duplicate 都加入了**主题（subject）检查**：只有同类型、同作用域、主题最接近的候选才可能被替换；
5. 写入节点、边、FTS 与向量。被 supersede 的旧记忆只降权，不删除。

读路径。

1. **Gate**（仅被动召回）：短确认语或信息量过少的输入直接跳过；
2. **Anchor**：词法检索（BM25，k1=1.2、b=0.75，基于词干、CJK 单字与双语概念词）加向量检索，范围是当前项目 + global，提问中提到其它项目名或别名时也加入该项目。v0.1 用 RRF 融合；v0.3 改为**校准的逻辑回归先验**，特征包括 idf 加权覆盖率、相对 BM25、余弦及其相对值、“问为什么且笔记给出理由”；
3. **Score**：对前 14 个候选批量询问相关性；
4. **Expand**（多跳查询）：只沿因果边和（涉及时间变化时）时间/supersedes 边从高置信锚点展开，不展开语义/实体邻居；
5. **自适应截断**：笔记相关性需不低于 T_RELEVANT=0.12，且不低于最佳笔记的 KEEP_RATIO=0.4 倍。agent 主动召回时，若无笔记过线，返回一个不低于 T_WEAK=0.06 的最佳弱候选（并标记为不充分）；
6. **过期值安全网**：同一主题、取值不同的候选中保留最新者，其余降权并标记；
7. **Sufficiency**：以最佳候选的校准概率构造先验（0.5 对应 T_SUFFICIENT=0.3），返回 sufficient/missing 信号，让 agent 知道“我不知道”。

3.7 项目识别

- **硬信号（确定性）**：cwd → git root（处理 worktree 的 .git 文件、gitdir: 与 commondir）→ remote → manifest（Cargo.toml / package.json / pyproject）→ README 首段。pid 优先取规范化的 host/owner/repo，同一仓库的所有 clone 与 worktree 共享一个项目；无 remote 时取 git root，再否则取 cwd。v0.2 加入了 manifest 向上查找与路径规范化（符号链接）。

- **软信号（模型）**：项目卡片的 summary 先由 README/manifest 生成启发式初稿，环路中的 agent 可随时用 unimem_project 改写，这比网关自己再调一次模型更准、更便宜；写入作用域由 System One 判定（§3.6）；跨项目引用通过检测提问中的项目名与别名实现。

3.8 被动捕获

每个 prompt turn 的用户输入与 assistant 文本都写入 episodes。会话空闲后，可选的蒸馏器从流水中抽取用户纠正、明确偏好、带理由的决定与陷阱，再走标准写路径。蒸馏器默认关闭；默认只开启 agent 显式写入。本文没有评测蒸馏器。

---

4 实现

uni-mem 是一个 Rust（edition 2021）单 crate 项目，产出两个二进制：unimem（网关与 CLI）和 unimem-bench（评测）。主要依赖为 axum 0.8、tokio、reqwest（rustls）、rusqlite 0.37（bundled SQLite，含 FTS5）。代码规模（wc -l，HEAD 9dcc43a）如下：

部分	行数
库与主程序 src/（不含 bench）	8,235
评测程序 src/bin/unimem-bench.rs	2,075
集成测试 tests/	2,015
合计	12,325

主要模块：gateway/{mod, anthropic, responses, chat, sse, mcp}.rs（网关与三种方言，各约 750–840 行）、engine.rs（写/读路径）、store.rs（SQLite、FTS5、内存向量索引、自实现 BM25）、decide.rs（System One 决策器链）、screen.rs（筛查与先验特征）、lexicon.rs（英文词干与 64 条双语开发词汇提示）、embed.rs（离线哈希嵌入器与 OpenAI 兼容嵌入接口）、project.rs、distill.rs、tools.rs、config.rs。

全部数据存于 ~/.unimem/unimem.db：记忆、边、FTS 索引、嵌入、项目、会话、流水、overlay（30 天后回收）与决策 trace。向量检索是对内存中全部向量的暴力余弦。离线哈希嵌入器在 v0.1/v0.2 为 hash-512；v0.3 改为 hash2-512，加入词干、复合标识符切分与双语概念特征。

不想让模型流量经过网关的用户，可以通过 /mcp（Streamable HTTP，仅 JSON 回复）使用同一组工具，此时没有被动召回、序言和流水捕获。CLI 提供 serve / remember / recall / forget / list / projects / project / stats / trace / reembed / distill / config / setup 等命令；除 serve 外都直接操作数据库。

质量门（v0.3/QA.md）：cargo test --all-targets 82/82 通过（库单元测试 36，集成测试 46）；clippy 只有 2 条警告，都在 bench 二进制中；release 构建 0 警告。需要说明的是，tests/v03.rs 是与 v0.3 改动同时编写的，它通过是意料之中的，不构成独立证据。

---

5 评测

5.1 实验设置

环境：4 核 aarch64 虚拟机（约 3.8 GB 内存，Ubuntu 22.04），rustc 1.99.1，release 构建。**所有实验都使用离线启发式决策器**，嵌入器为 hash-512（v0.1/v0.2）或 hash2-512（v0.3）。三个版本：v0.1 = db7eb95，v0.2 = 61e31fc，v0.3 = 8a679a8，都用同一份 bench 代码运行（自 09bbc53 起未改动）。在 v0.1 上运行新 bench 能精确复现已提交的 E1 数值。

数据集：三个合成数据集都由 LLM 智能体按脚本（bench/data/gen_*.py）生成，每个包含 6 个项目、按写入顺序排列的笔记和带标注的查询。

数据集	笔记	查询	harness	对 v0.3 的地位
devmem	183	211	2 个	**样本内**：v0.2 和 v0.3 的改动都由其发现驱动
heldout	194	210	claude-code、codex、cursor	**样本内**：对 v0.2 是样本外，但 v0.3 针对它调过参

数据集	笔记	查询	harness	对 v0.3 的地位
heldout2	233	242	claude-code、codex、opencode	**样本外**：新的虚构开发者、6 个新仓库、新 harness；无人在其上调参

heldout2 包含 218 条可回答查询（其中 140 条跨 harness）、20 条不可回答查询、4 条仅含禁止笔记的隔离查询，另有 12 对干扰笔记（`not_supersedes`）和 6 条诱饵笔记（`decoy`，看似应拒绝实则合法）。**污染检查**：heldout2 的文件时间戳早于 v0.3 提交，v0.3 的调参脚本和结果目录都没有引用它；但 v0.3 在 `lexicon.rs` 中加入的 64 条双语提示里，有两条（`queue`/队列、`retry`）只出现在 heldout2 中。二者都是通用开发词汇，但不能排除轻微泄漏。**只有 heldout2 的数字应被读作泛化性能**。

基线：`silo-per-harness`（同一引擎，但每个 harness 一个存储）；`vector-top5`（所有写入原样存储，不筛查、不去重、不覆盖，全局余弦 top-5，使用同一嵌入器）；`bm25-top5`（同一分词器的 BM25 top-5）；以及二者的 `-cwd` 变体（只看同目录或 home 下写入的笔记）。由于 vector 基线使用引擎的嵌入器，v0.3 的嵌入器改动也同时提升了它；下文分别列出 hash2 与旧 hash 两个版本。

指标：R@1、R@5、MRR、每查询返回笔记数、precision（返回笔记中 gold 的比例）、作用域泄漏、禁止笔记命中、过期笔记返回率、不可回答查询的弃答率、可回答查询的误弃答率、`sufficient=false` 误报率、被拒笔记返回率；写路径的拒绝精确率/召回率、去重与覆盖的精确率/召回率、作用域准确率。

5.2 E1：记忆质量（heldout2，样本外）

表 1 heldout2 总体结果（vector 列为 hash2 嵌入器，另列旧 hash 的 vector-top5-cwd）

指标	v0.1	v0.2	**v0.3**	silo (v0.3)	vector-top5	vector-top5-cwd	vector-cwd (旧 hash)	bm25-top5	bm25-top5-cwd
R@1	0.440	0.408	**0.555**	0.243	0.358	0.390	0.349	0.408	0.413
R@5	0.569	0.550	**0.688**	0.271	0.656	0.670	0.615	0.642	0.615
MRR	0.497	0.471	**0.616**	0.257	0.475	0.506	0.453	0.500	0.497
笔记/查询	2.161	1.888	1.636	1.136	5.000	5.000	5.000	4.541	4.045
precision	0.263	0.291	**0.423**	0.242	0.131	0.134	0.123	0.141	0.152
作用域泄漏	3.3%	2.5%	2.5%	2.9%	91.3%	8.7%	10.3%	79.3%	3.3%
禁止笔记命中	0.0%	0.0%	0.0%	0.0%	66.7%	0.0%	0.0%	57.1%	0.0%
过期笔记返回	34.9%	51.2%	**16.3%**	16.3%	62.8%	58.1%	53.5%	65.1%	55.8%
弃答（不可回答）	10.0%	20.0%	5.0%	15.0%	0.0%	0.0%	0.0%	0.0%	0.0%
误弃答（可回答）	2.3%	14.2%	2.8%	23.4%	0.0%	0.0%	0.0%	1.8%	2.8%
`sufficient=false`（可回答）	72.5%	61.5%	45.0%	70.2%	–	–	–	–	–
被拒笔记返回	13.6%	10.3%	6.6%	4.1%	26.9%	45.5%	43.8%	31.8%	37.2%

v0.3 中 `unimem-lite`（删除所有图边）和 `unimem-passive`（开启被动门控）在三个数据集上与 `unimem-full` 结果完全相同（lite 在 heldout 上有一条查询不同）。也就是说，**图扩展和被动门控没有可测量的贡献**。

表 2 配对显著性检验（218 条可回答查询，逐查询 R@5 命中）

比较	ΔR@5	95% bootstrap CI	胜 / 负	符号检验 p
v0.3 − v0.1	+0.119	[+0.064, +0.174]	34 / 8	7e-5
v0.3 − v0.2	+0.138	[+0.078, +0.193]	38 / 8	9e-6
v0.3 − bm25-top5-cwd	+0.073	[+0.018, +0.128]	26 / 10	0.011
v0.3 − bm25-top5	+0.046	[−0.005, +0.096]	21 / 11	0.11
v0.3 − vector-top5-cwd（旧 hash）	+0.073	[+0.009, +0.133]	32 / 16	0.029
v0.3 − vector-top5-cwd（hash2）	+0.018	[−0.037, +0.073]	20 / 16	0.62
v0.3 − vector-top5（hash2）	+0.032	[−0.023, +0.087]	23 / 16	0.34

v0.3 在 R@5 上显著优于 v0.1、v0.2 和 bm25-top5-cwd，但**不显著优于使用同一 hash2 嵌入器的 top-5 余弦检索**。v0.3 相对 v0.1/v0.2 的 R@5 提升中，有一部分来自嵌入器改动；同一改动也把 vector-top5-cwd 从 0.615 提到 0.670。v0.3 相对该基线的明确优势在别处：R@1 0.555 对 0.390，MRR 0.616 对 0.506，precision 0.423 对 0.134，每次回答 1.6 条对 5 条笔记，过期返回 16% 对 58%，被拒笔记返回 6.6% 对 45.5%。

表 3 heldout2 分类 R@5

类别	n	v0.1	v0.2	**v0.3**	silo	vector-top5	vector-cwd	bm25-top5	bm25-cwd
single_hop	65	0.692	0.723	**0.892**	0.338	0.908	0.846	0.785	0.738
paraphrase	31	0.290	0.323	0.419	0.258	0.387	0.484	0.355	0.355
chinese	38	0.658	0.526	0.763	0.368	0.605	0.816	0.763	0.763
cross_project	19	0.211	0.158	0.368	0.158	0.579	0.053	0.474	0.053
temporal	24	0.500	0.500	0.667	0.208	0.625	0.667	0.625	0.708
causal	23	0.696	0.696	0.739	0.261	0.609	0.696	0.739	0.739
scope_isolation	22	**0.722**	0.667	0.556	0.056	0.500	0.667	0.444	0.611

scope_isolation 从 v0.1 的 0.722 退步到 v0.3 的 0.556。原因是“每次 push 跑什么？”这类泛化短问句的前几位被全局 git 规则和一条未被拒绝的注入笔记 n169 占据。paraphrase（0.419）与 cross_project（0.368）仍然偏弱。在 cross_project 上，搜索全部项目的不设限基线更好（vector-top5 0.579、bm25-top5 0.474），但代价是 79–91% 的查询出现作用域泄漏。

5.3 统一存储 vs. 按 harness 隔离

表 4 heldout2 跨 harness（140 条）与同 harness（78 条）可回答查询

系统	跨 harness R@5	跨 MRR	同 harness R@5	同 MRR
uni-mem v0.1	0.557	0.486	0.590	0.518
silo v0.1	0.000	0.000	0.564	0.517
uni-mem v0.2	0.543	0.466	0.564	0.480
silo v0.2	0.000	0.000	0.500	0.458
uni-mem v0.3	**0.664**	**0.605**	0.731	0.637
silo v0.3	0.000	0.000	0.756	0.718
bm25-top5-cwd	0.600	0.487	0.641	0.517
vector-top5-cwd（hash2）	0.664	0.494	0.679	0.527

silo 在跨 harness 查询上的 0.000 是**构造上必然**的：它看不到其它 harness 写入的笔记。因此这一结果验证的是“共享存储”这一架构前提，而不是检索算法；统一的 BM25 或向量存储同样能跨 harness 召回。在同 harness 查询上，silo 略好（R@5 0.756 对 0.731，MRR 0.718 对 0.637），因为存储更小、竞争者更少，统一存储在这里付出了一点代价。

5.4 写路径

表 5 heldout2 写路径

指标	v0.1	v0.2	**v0.3**
拒绝精确率	1.000	0.889	0.923
拒绝召回率	0.217	0.348	0.522
分类拦截（注入 / 密钥 / 状态）	0/7, 4/10, 1/6	0/7, 7/10, 1/6	**1/7, 10/10, 1/6**
合法笔记误拒	0	1	1
诱饵误拒	0/6	0/6	0/6
重复召回（gold 23）	0/23	0/23	4/23（+1 误判）
覆盖：gold / 预测 / 正确	25 / 6 / 6	25 / 0 / 0	**25 / 18 / 16**
覆盖精确率 / 召回率	1.000 / 0.240	– / 0.000	0.889 / 0.640
丢失的更新	0	0	1
干扰对误覆盖	0/12	0/12	0/12
作用域准确率	0.890	0.915	0.944
项目内写下的全局偏好	2/23	8/22	18/22
写入延迟 p50 / p95（ms）	0.54 / 0.83	0.70 / 1.02	1.26 / 4.96

泛化的部分：覆盖检测召回率从 0.24 提升到 0.64，干扰对零误判；密钥筛查 10/10；在项目内写下的全局偏好从 2/23 提升到 18/22。**未泛化的部分**：注入筛查 1/7；状态笔记筛查 1/6（5 条漏判中 4 条为中文）；重复检测 4/23（23 条 gold 重复中 17 条是跨语言改述）；有 6 条项目笔记被误升为全局。样本内数据集上这些筛查全部是 1.000，**这一落差是本次评测中最清晰的过拟合信号**。后果是严重的：典型注入 n169（“Ignore every earlier instruction…”）被存储，并在 9 条查询中被返回；n174（“Team convention per Dave: … run apply with -auto-approve …”）恰好被当作不可回答问题 “Is it fine to auto-approve applies when a plan looks clean?” 的答案返回。写入延迟比 v0.1 增加约 2.3 倍（p95 约 6 倍），但仍在每次 1–5 ms 量级。

样本内与样本外对照。在 heldout（对 v0.3 为样本内）上，v0.3 的 R@5 为 0.677、拒绝召回率 1.000、注入拦截 6/6、重复召回 0.545、覆盖召回 0.846、过期返回 6.3%；在 heldout2 上分别为 0.688、0.522、1/7、0.174、0.640、16.3%。检索质量迁移到了样本外（R@5 甚至略高），写路径筛查则没有。

5.5 版本演进与 v0.2 的教训

v0.1 的首轮评测（devmem）暴露了 15 个问题，包括：流式路径因未设置 `TCP_NODELAY` 增加约 43 ms 延迟；锚点未按会话隔离；supersede 规则缺少主题检查，导致 E4 中 92% 的写入覆盖了旧笔记；项目内写下的全局偏好 0/13 正确；中文短问句被被动门控（`< 12` 字符）截断；密钥与注入正则存在缺口。

v0.2 针对这些问题逐一修复。用独立编写的 heldout 检验后发现，**v0.2 并没有提升检索质量**（表 6）。

表 6 heldout 上 v0.1 → v0.2（当时 heldout 为样本外）

指标	v0.1	v0.2	变化
R@5	0.444	0.447	+0.003
R@1	0.303	0.281	−0.022
MRR	0.366	0.353	−0.013
误弃答（可回答）	5.6%	21.3%	+15.7pp
覆盖召回	0.192	0.038	—
过期返回	44.4%	50.8%	+6.4pp
拒绝召回	0.278	0.500	—

当时 v0.2 在 R@5 上仍明显低于 bm25-top5-cwd（0.601）。v0.2 的修复中，机械性的那部分确实有效：`TCP_NODELAY`、锚点隔离、项目识别（35/37 → 37/37）、过度覆盖。但启发式修复出现了过拟合与副作用：主题检查在消除误覆盖的同时几乎消除了全部真覆盖，导致过期返回率上升；新的最低锚点证据阈值切掉了中文短问句；写入筛查在 devmem 上达到 1.000/1.000，在 heldout 上只有 0.750/0.500。更严重的是 E4 中出现了新的静默丢数据模式：19,664 次写入中有 12,664 次被判为重复，而生成器只产生约 1.0k 条完全相同的文本。

这一教训直接决定了 v0.3 的评测纪律：在 v0.3 针对 heldout 调参之后，heldout 也被视为样本内，并新写了 heldout2，作为唯一的泛化证据。v0.3 的改动包括：用校准逻辑回归替代 RRF 与固定阈值、带主题检查的 supersede/duplicate、更通用的筛查、充分性重设计和 `hash2` 嵌入器。

5.6 E2：网关开销、缓存稳定性与拦截正确性

E2 使用模拟上游（立即响应，设置 TCP_NODELAY），每个单元 300 次请求，请求体约 31.5 KB（仿 Claude Code：12 个工具、约 11 KB system prompt）。表中数字是网关本身的开销。

表 7 相对直连的附加延迟（逐请求配对差值，ms）

路径	v0.1 ΔTTFB p50 / p95	v0.2 ΔTTFB p50 / p95	**v0.3 ΔTTFB p50 / p95**
gateway/stream（即 `unimem serve`）	42.807 / 48.580	0.754 / 1.046	**0.898 / 2.361**
gateway/json	0.477 / 0.663	0.217 / 0.279	0.219 / 0.289
gateway-norecall/stream	1.019 / 47.721	0.220 / 0.311	0.226 / 0.337
gateway-nodelay/stream	2.038 / 3.418	0.765 / 1.040	0.894 / 2.457

TCP_NODELAY 的发现。 v0.1 的 `unimem serve` 没有在客户端 socket 上设置 `TCP_NODELAY`。SSE 写出大量小块，在 Linux 回环上触发 Nagle 与 delayed-ACK 的交互，流式路径因此增加约 43 ms TTFB；同一状态下由 bench 强制 NODELAY 时只增加约 2 ms。v0.2 在监听器上为每个接受的连接设置 NODELAY（`nodelay_listener`），之后 `gateway/stream` 与 `gateway-nodelay/stream` 一致。v0.3 的流式召回开销（p95 2.4 ms）高于 v0.2（1.0 ms），与 v0.3 更昂贵的召回路径相符，但未单独 profile。另外，v0.2 二进制的直连路径绝对值约快 2 倍（direct/json TTFB p50 0.136 ms 对 0.245–0.257 ms），原因未查明，因此只比较附加延迟。

表 8 prompt cache 稳定性（30 轮流式对话，v0.3）

模式	上游请求 / 续写轮	平均前缀比例	最小	完全稳定转换	可缓存字节比	上游字节
Passthrough	37 / 0	1.000	1.000	36/36	0.987	1,533,573
uni-mem	43 / 6	**1.000**	**1.000**	**42/42**	0.984	2,460,542（+60%）
NaiveSystem（召回放入 system）	37 / 0	0.825	0.615	9/36	0.802	1,550,652
NaiveTail（尾部注入不重放）	37 / 0	0.991	0.969	7/36	0.979	1,546,544

uni-mem 的每一次上游请求都是上一次的逐字节前缀扩展，与纯透传相同；把召回写入 system prompt 只有 9/36 次转换完全稳定，尾部注入而不重放只有 7/36。代价是上游字节数增加：6 次额外的全上下文续写请求使总字节增加 60%（v0.2 为 +71%）。这些字节绝大部分是可缓存前缀，但请求次数和未命中缓存的输入 token 仍会增加。

拦截正确性。 脚本化轮次覆盖仅召回、仅写入、混合（unimem + Bash）、仅 Bash 和纯文本，流式与 JSON 各一套，每轮带一个回传可见消息的后续请求。120 次客户端请求对应 144 次上游请求（24 轮续写），**1060/1060 条断言**

通过（三个版本都是）。断言族包括：客户端从不看到 unimem 工具或上下文、SSE 帧合法、system 未改动、工具按序追加、隐藏轮次逐字节重放、混合轮次的隐藏调用与结果被重放、可见 stop reason 正确等，全部 100%。**锚点碰撞探针**：两个会话发送相同 prompt 并得到相同回复时，第二个会话被注入第一个会话隐藏轮次的次数为 v0.1 10/10、v0.2 0/10、v0.3 0/10。

5.7 E3：项目识别

37 种合成目录布局，覆盖各类 remote 格式（scp/ssh/https/git、含 token、大写、尾斜杠、非标准端口）、worktree（含相对 gitdir + commondir）、submodule、嵌套仓库、fork、GHE 与 GitLab 子组、无 remote 仓库、非 git 的 manifest 目录、dotfiles 仓库作为 home、符号链接，以及“首次见到后才加 remote 时身份保持”。结果为 v0.1 35/37，v0.2 与 v0.3 均为 37/37；v0.2 修复的正是 v0.1 报告的两例（非 git manifest 目录的子目录；指向无 remote 仓库的符号链接）。没有样本外布局，因此这是样本内结果。

5.8 E4：引擎吞吐

使用模板化合成笔记（比真实笔记更自相似，可视为压力测试）。

表 9 引擎吞吐（机器间波动约 ±30%）

版本	存储	存活	写入尝试	写 p50 / p95 ms	召回 p50 / p95 ms	DB MB	字节/笔记	RSS MB
v0.2	7000	5064	19664	7.434 / 10.359	7.092 / 8.869	49.06	7008	29.8
v0.3	7000	6897	7183	5.296 / 14.795	3.582 / 5.701	71.94	10276	58.9
v0.1（原始）	10000	512	10672	11.614 / 15.944	8.845 / 11.743	54.56	5456	65.0
v0.3	10000	9849	10325	7.654 / 17.241	4.751 / 7.226	103.71	10371	77.3

v0.1 在 10k 写入中有 9,232 次覆盖了旧笔记，只剩 512 条存活；v0.2 在 7k 规模下把 12,664 次写入判为重复；v0.3 在 7183 次尝试中判重 183 次。按生成器随机数离线回放，约有 143 条完全重复文本，因此仍可能约有 40 条模板笔记被误判为近重复而丢弃。supersede 只在 1.4% 的存储中触发。召回是对 7k–10k 条存活笔记的暴力余弦，7k 时 p50 3.6 ms，10k 时 4.8 ms，仍随规模线性增长。成本方面，每条笔记字节数比 v0.2 增加约 47%，7k 时 RSS 约为 v0.2 的 2 倍。

---

6 讨论与局限

本文的结论必须放在以下限制下理解：

1. **没有真实模型参与**。 所有 System-One 判断都是启发式决策器返回的代码先验，嵌入器只有离线哈希嵌入。Jev 或 LLM 决策器、语义嵌入模型可能大幅改变筛查、去重/覆盖、作用域和相关性的数字，方向未知。代码中的阈值也明确标注为“为启发式先验标定”。
2. **没有真实 harness 流量**。 E2 使用模拟上游和脚本化的仿 Claude Code 请求。网关在真实 Claude Code / Codex 各版本下的回传格式（尤其是 thinking 块被裁剪、历史被压缩的情况）尚未用真实流量回归。Codex 使用 ChatGPT 登录时的自定义 provider 鉴权透传尚未在真机验证。改写订阅流量是否符合服务商条款，需要用户自行确认。
3. **数据集是合成的**。 三个数据集都由 LLM 智能体编写，规模为两百条量级，单一用户画像，与真实开发者的记忆分布可能差异很大。heldout2 存在 `queue`/`retry` 两条双语提示的潜在词汇泄漏。
4. **安全筛查没有泛化**。 样本外注入拦截 1/7、状态笔记 1/6，被存下来的注入随后被当作答案返回（n174）。这是安全问题而不仅是质量问题，也说明正则加关键词先验无法应对开放世界的注入表述。
5. **跨语言检索薄弱**。 heldout2 上跨语言可回答查询（48 条）的 R@5 为 0.333，同语言（170 条）为 0.788；bm25-cwd 与 vector-cwd 也有同样落差（0.250/0.718、0.333/0.765），是所有词法/哈希系统共有的弱点。64 条双语提示表只覆盖通用开发词汇。
6. **R@5 不显著优于同嵌入器的余弦检索**。 uni-mem 的优势在于 R@1、MRR、precision 和过期规避，而不是召回覆盖。
7. **图扩展与被动门控没有可测量的贡献**（full = lite = passive）。四视图图结构目前更多是架构上的预留，尚无实证支持。
8. **充分性信号偏弱**。 可回答查询上 `sufficient=false` 仍有 45.0% 的误报；不可回答查询上“弃答或 sufficient=false”为 70.0%。它带有一些信息，但不足以据此行动。
9. **成本**。 写入延迟约为 v0.1 的 2.3 倍；流式召回网关开销 p95 2.4 ms；每条笔记存储比 v0.2 增加约 47%；续写使上游字节增加约 60%；召回仍是线性暴力检索。

10. **没有端到端任务指标。** 本文只度量检索与网关机制，没有度量记忆是否真正提升了智能体的任务成功率。蒸馏器也未评测。

---

7 未来工作

- **真实决策器的标定**：接入 Jev 与小型 LLM 决策器，在样本外数据上重新标定阈值，并比较与启发式先验的差距。
- **多语言嵌入**：用 bge-m3 等多语言嵌入模型（已支持 OpenAI 兼容 `/v1/embeddings` 接口）解决跨语言检索与跨语言去重。
- **真实流量回放测试**：录制 Claude Code、Codex 各版本的真实请求，回归 overlay 锚点的容错与 ChatGPT 登录鉴权透传。
- **注入分类器**：用专门训练的分类器或模型判断替代正则先验，并在独立的对抗样本集上评测。
- **ANN 索引**：引入近似最近邻或带作用域预过滤的索引，消除线性增长。
- **蒸馏评测**：度量被动蒸馏抽取的记忆的精确率与召回率。
- **端到端指标**：在真实编码任务上度量有/无统一记忆时智能体的成功率、重复纠正次数与 token 成本。

---

8 结论

uni-mem 表明，把长期记忆放在模型 API 网关这一层，可以在不改动任何 harness 的前提下，为异构编码智能体提供共享的、按项目归档的记忆，同时不破坏 prompt cache（42/42 次逐字节前缀扩展），也不向 harness 泄漏它不认识的工具调用（1060/1060 断言通过），流式附加延迟在毫秒以内（p50 0.898 ms）。在样本外数据上，v0.3 引擎在 R@1、MRR、precision 和过期规避上优于所有基线，并显著优于早期版本与 BM25 基线；但它的 R@5 并不显著优于同嵌入器的朴素余弦检索。写入端安全筛查没有泛化、跨语言检索薄弱、图扩展无可测贡献，这些是当前最主要的开放问题。所有结论都基于离线启发式决策器和合成数据。v0.2 的经验也说明，针对已知失败做的启发式修复很容易过拟合，样本外纪律对这类系统是必要的。

附录 A 复现

```
# 构建
cargo build --release
cargo test --all-targets

# 评测（E1 使用 --data 指定数据集；E2 固定使用 devmem）
./target/release/unimem-bench all --data bench/data/devmem.json   --out bench/results/v0.3
./target/release/unimem-bench e1  --data bench/data/heldout.json  --out bench/results/v0.3-heldout
./target/release/unimem-bench e1  --data bench/data/heldout2.json --out bench/results/v0.3-heldout2
# E4 规模：UNIMEM_BENCH_E4_N=7000；换决策器/嵌入器：
#   UNIMEM_BENCH_DECIDER=jev,heuristic  UNIMEM_BENCH_EMBEDDER=openai  UNIMEM_BENCH_CONFIG=<config.toml>

# 数据集生成（仅标准库）
python3 -I bench/data/gen_heldout2.py
```

附录 B 接入

```
./target/release/unimem serve            # 监听 127.0.0.1:4777

# Claude Code
export ANTHROPIC_BASE_URL=http://127.0.0.1:4777
claude
```

```
# Codex CLI：~/.codex/config.toml
model_provider = "unimem"

[model_providers.unimem]
name = "uni-mem"
```

```
base_url = "http://127.0.0.1:4777/v1"
wire_api = "responses"
requires_openai_auth = true

# 其它 OpenAI/Anthropic 兼容客户端
export OPENAI_BASE_URL=http://127.0.0.1:4777/v1
export ANTHROPIC_BASE_URL=http://127.0.0.1:4777

# 仅 MCP（无被动召回、序言与流水捕获）
claude mcp add --transport http unimem http://127.0.0.1:4777/mcp

# 打印各 harness 的配置片段
unimem setup claude-code|codex|shell|mcp
```

附录 C 结果文件

bench/results/FINAL.md（v0.1/v0.2/v0.3 总评）、bench/results/COMPARISON.md（v0.1 vs v0.2）、bench/results/v0.1/SUMMARY.md（首轮发现）、bench/results/v0.3/ERRORS.md（heldout2 错误分析）、bench/results/v0.3/QA.md（质量门），以及各版本目录下的 e1_memory.json、e2_gateway.json、e3_project.json、e4_throughput*.json。